

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 07 -

SELEÇÃO DE MODELOS E VALIDAÇÃO CRUZADA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	7
MERCADO, CASES E TENDÊNCIAS	17
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS.....	25

EMSE

O QUE VEM POR AÍ?

Imagine que você desenvolveu um modelo de Machine Learning super sofisticado e o treinou nos seus dados de treino — ele acerta praticamente tudo que viu! Seria tentador já “vendê-lo” como solução definitiva. Porém, como ter certeza de que ele vai manter esse desempenho em dados novos? Assim como lançar uma versão de software sem testes adequados pode causar surpresas desagradáveis aos usuários, selecionar um modelo de aprendizado sem validação rigorosa pode levar a erros custosos quando ele for utilizado no mundo real.

Nesse cenário entra em cena a validação cruzada (cross-validation), um conjunto de técnicas estatísticas para estimar de forma mais confiável a performance de modelos em dados não vistos. Em vez de treinar apenas uma vez e avaliar em um único conjunto de teste (o que pode dar sorte ou azar dependendo da partição escolhida), a validação cruzada permite avaliar o modelo várias vezes em diferentes partições dos dados. Outra vantagem: conseguimos aproveitar melhor os dados disponíveis — todos os exemplos são usados para treinar (em $K-1$ das rodadas) e para validar (em 1 das rodadas), diferente do simples treino/teste onde a divisão fixa deixa parte dos dados só para teste.

Nesta aula, vamos entender por que essa abordagem se tornou fundamental para escolher modelos em projetos de IA modernos e como aplicá-la na prática para comparar modelos e ajustar hiperparâmetros. Veremos que a validação cruzada, embora um pouco mais custosa computacionalmente (afinal, treinamos múltiplas vezes), funciona como um “amortecedor” contra conclusões enganosas. Ela ajuda a evitar que escolhamos o modelo errado por causa de acaso ou particularidades de um único conjunto de validação. Ao longo do caminho, discutiremos as armadilhas de avaliações malfeitas, exploraremos variações do método (como LOO e validação em séries temporais) e aprenderemos boas práticas para conduzir experimentos de modelagem de forma confiável. Ao fim, você estará equipado(a) para selecionar modelos de Machine Learning com confiança, assim como um(a) bom(a) engenheiro de software gerencia versões com testes e deploys incrementais em produção — minimizando surpresas e mantendo o controle de qualidade.

HANDS ON

Vamos praticar a seleção de modelos usando validação cruzada em um exemplo simples. Suponha que queiramos ajustar o grau de um modelo polinomial para se adaptar a um conjunto de pontos. Geraremos dados sintéticos de um polinômio cúbico com ruído e tentaremos descobrir qual grau de polinômio se ajusta melhor, sem cair no sobreajuste. Em vez de simplesmente treinar modelos de vários graus e escolher o que tiver menor erro no mesmo conjunto de treino (o que não seria correto), usaremos 5-fold cross-validation para estimar o erro de generalização de cada modelo. Assim, esperamos identificar o grau que generaliza melhor para dados novos.

No código a seguir, primeiro criamos os dados e depois definimos uma função `cross_val_mse` que realiza a validação cruzada calculando o erro quadrático médio (MSE) de um modelo polinomial de grau dado. Em seguida, iteramos sobre alguns graus candidatos e calculamos seus MSEs médios de validação, escolhendo o melhor. Observar os resultados deve revelar que modelos de grau muito alto obtêm erro médio pior (devido a sobreajuste), enquanto um grau intermediário deve minimizar o erro. Os comentários no código explicam cada passo:

```
1     import numpy as np
2
3     # 1. Dados sintéticos:  $y = 1 + 0.5x - 2x^2 + 0.3x^3$ 
+ ruído
4     np.random.seed(42)
5     N = 60
6     X = np.linspace(-3, 3, N).reshape(-1, 1)
# feature unidimensional
7     y_true_fn = lambda x: 1 + 0.5*x - 2*x**2 + 0.3*x**3
# polinômio cúbico verdadeiro
8     y = y_true_fn(X) + np.random.normal(scale=5.0,
size=X.shape) # adiciona ruído
9
10    # Embaralha os índices para depois dividir em folds
11    indices = np.random.permutation(N)
12    K = 5
13    folds = np.array_split(indices, K) # divide
índices em 5 partes aproximadamente iguais
1     # 2. Função para realizar K-fold cross-val e
retornar MSE médio de um modelo polinomial de certo grau
2     def cross_val_mse(degree, K=5):
3         errors = []
```

```

4         for k in range(K):
5             # Separa o fold k como validação, o resto
como treino
6                 val_idx = folds[k]
7                 train_idx = np.concatenate([folds[i] for i
in range(K) if i != k])
8                 X_train, y_train = X[train_idx],
y[train_idx]
9                 X_val, y_val = X[val_idx], y[val_idx]
10                # Ajusta um polinômio de grau 'degree'
usando numpy.polyfit (regressão linear polinomial)
11                coeffs = np.polyfit(X_train.ravel(),
y_train.ravel(), degree)
12                # Avalia no conjunto de validação
13                y_pred = np.polyval(coeffs, X_val.ravel())
14                mse_val = np.mean((y_val.ravel() -
y_pred)**2)
15                errors.append(mse_val)
16                return np.mean(errors)
1 # 3. Avalia modelos de diferentes graus usando a
função de cross-val e escolhe o melhor
2     degrees = [1, 2, 3, 4, 5, 6, 7] # candidatos para
grau do polinômio
3     best_deg = None
4     best_mse = float("inf")
5     for deg in degrees:
6         mse = cross_val_mse(deg, K=5)
7         print(f"Modelo grau {deg}: MSE médio =
{mse:.2f}")
8         if mse < best_mse:
9             best_mse = mse
10            best_deg = deg
11
12    print(f"\nMelhor modelo: grau {best_deg} (MSE médio
{best_mse:.2f})")

```

Código-fonte 1 - import numpy as np
 Fonte: Elaborado pelo autor (2026)

Rodando o código, você verá algo parecido com:

```

1     Modelo grau 1: MSE médio = 171.29
2     Modelo grau 2: MSE médio = 26.07
3     Modelo grau 3: MSE médio = 25.52
4     Modelo grau 4: MSE médio = 25.60
5     Modelo grau 5: MSE médio = 25.74
6     Modelo grau 6: MSE médio = 26.80
7     Modelo grau 7: MSE médio = 28.45
8
9     Melhor modelo: grau 3 (MSE médio 25.52)

```

Código-fonte 2 – Resultado do código (Python)
 Fonte: Elaborado pelo autor (2026)

Ou seja, o modelo polinomial de grau 3 – justamente a ordem original dos dados – obteve o menor erro médio de validação, sendo escolhido como o melhor. Note que o grau 2 ficou bem próximo (25.52 vs 26.07) nos dados gerados; graus 4 e 5 também tiveram desempenho similar, diferença pequena em relação ao cúbico. Já modelos mais simples (grau 1) subajustaram com erro médio alto, enquanto modelos muito complexos (grau 6, 7) começaram a piorar o erro médio, sugerindo sobreajuste.

Em problemas reais, observar um aumento do erro de validação em modelos mais complexos é um forte indicativo de que adicioná-los traz mais variância do que ganho – exatamente o que a validação cruzada nos ajuda a detectar antes de fazer um deploy de um modelo inadequado em produção. Caso tivéssemos apenas avaliado pelo erro no treino, graus altos pareceriam melhores (eles podem atingir MSE de treino praticamente zero se permitidos); mas graças à validação cruzada pudemos enxergar além do desempenho nos dados de treino e escolher o modelo com melhor equilíbrio entre erro e complexidade.

SAIBA MAIS

Vimos na prática como a validação cruzada orienta a seleção de modelos, evitando escolher um polinômio muito complexo que pareceria bom apenas no treino. Agora é hora de dissecar esses conceitos de forma mais ampla e teórica. Nesta seção, vamos entender por que precisamos de métodos como cross-validation, formalizar o problema de seleção de modelos e discutir diferentes estratégias de validação e tuning, além de situar isso em um contexto maior de aprendizado (incluindo aspectos estatísticos e computacionais).

O dilema do ajuste: underfitting vs overfitting

Todo problema de seleção de modelos envolve equilibrar dois extremos: de um lado, modelos simples podem subajustar (underfit), não capturando padrões importantes; de outro, modelos complexos demais podem sobreajustar (overfit), capturando não só o padrão mas também ruídos ou coincidências específicas dos dados de treino. Em termos práticos, sobreajuste significa que o modelo tem desempenho excelente nos dados que viu, mas falha em generalizar para novos dados. Formalmente, podemos pensar que um modelo pertence a uma família $\mathcal{H}$ (hipóteses possíveis) parametrizada por algum parâmetro de complexidade (por exemplo, grau do polinômio, número de neurônios etc.).

Queremos encontrar $h \in \mathcal{H}$ que minimize o erro de generalização – isto é, o erro esperado em dados novos – mas só temos acesso a um conjunto finito de dados de treinamento D . Se usarmos o erro em D (erro empírico) como critério único, modelos mais complexos tendem a ter erro empírico zero (eles podem memorizar D inteiro, se flexíveis o suficiente). Contudo, o verdadeiro objetivo é minimizar o erro em novos dados. A decomposição bias-variance nos ajuda a entender isso: o erro esperado de um modelo pode ser dividido em três termos:

$$E \left[(y - \hat{f}(x))^2 \right] = \underbrace{\text{Bias}[\hat{f}(x)]^2}_{\text{erro de viés}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{erro de variância}} + \underbrace{\sigma^2}_{\text{ruído irreduzível}},$$

onde y é o verdadeiro valor e $\hat{f}(x)$ a predição do modelo. Modelos muito simples têm viés alto (não conseguem captar a estrutura correta, errando sistematicamente), enquanto modelos muito complexos tendem a ter variância alta (se ajustam aos detalhes aleatórios da amostra de treino, variando muito se mudarmos os dados). O termo de ruído σ^2 é inerente ao problema (não podemos eliminá-lo, pois representa fatores aleatórios nos dados). Portanto, ao aumentar a complexidade do modelo, geralmente reduzimos o viés mas aumentamos a variância. Existe um ponto ótimo (compromisso viés-variância) onde o erro total é mínimo – e o papel da validação é justamente identificar esse ponto na prática.

Seleção de modelos como otimização

Podemos formular a seleção de um modelo ou ajuste de hiperparâmetros como um problema de otimização:

$$\min_{h \in \mathcal{H}} (E_{\text{teste}}(h))$$

onde $E_{\text{teste}}(h)$ denota o erro verdadeiro do modelo h (em distribuição de teste). É comum aproximarmos E_{teste} pelo erro de validação E_{val} medido via cross-validation ou um conjunto hold-out. Em alguns casos, adicionamos restrições de complexidade ou penalizações: por exemplo, resolver

$$\min_{h \in \mathcal{H}} (\text{Big}(E_{\text{val}}(h) + \alpha \cdot \Omega(h))\text{Big}),$$

onde $\Omega(h)$ é uma medida de complexidade do modelo (como número de parâmetros) e α um peso. Critérios como AIC e BIC seguem essa filosofia penalizando modelos com muitos parâmetros para tentar compensar o sobreajuste. Por exemplo, o Akaike Information Criterion define:

$$\text{AIC} = -2 \ln(\mathcal{L}) + 2p,$$

onde $\ln(\mathcal{L})$ é o log-verossimilhança do modelo e p o número de parâmetros estimados. Minimizar AIC busca modelos com bom ajuste ($-2\ln\mathcal{L}$ baixo) mas também poucos parâmetros ($2p$). Ainda que úteis, esses critérios fazem suposições aproximadas e nem sempre capturam todas as nuances dos dados modernos – por isso, na prática de Machine Learning, confiamos muito em validação cruzada empírica para comparar modelos.

Infelizmente, resolver exatamente o problema evidenciado (encontrar o melhor modelo no espaço de todas as arquiteturas e configurações) é geralmente intratável. Se nosso espaço de hipóteses é grande (hiperparâmetros contínuos, arquiteturas complexas), não há fórmula fechada – precisamos buscar. Pior: testar todas as combinações de hiperparâmetros cresce explosivamente conforme adicionamos dimensões. Por exemplo, se você quer explorar cinco valores para cada um de três hiperparâmetros, isso já dá $5^3 = 125$ modelos a treinar e validar. Em geral, com d hiperparâmetros e k_i valores para o i -ésimo, o total de configurações do grid é

$$N_{\text{config}} = \prod_{i=1}^d k_i.$$

Esse número explode facilmente. De fato, há resultados teóricos indicando que encontrar o conjunto ótimo de hiperparâmetros ou a estrutura ótima de um modelo pode ser NP-difícil (intuitivamente, tão difícil quanto resolver problemas combinatórios complexos). Por isso, frequentemente recorremos a heurísticas de busca em vez de exaustão completa. A abordagem mais simples é o grid search (escolher um grid de valores e testar todos, como no exemplo anterior).

Uma alternativa muitas vezes mais eficiente é o random search, que realiza amostras aleatórias no espaço de hiperparâmetros; surpreendentemente, random search pode encontrar bons parâmetros com menos tentativas que grid, principalmente quando alguns hiperparâmetros não influenciam tanto o resultado. Métodos mais avançados incluem busca bayesiana (por exemplo, usando processos Gaussianos ou florestas aleatórias para modelar a função de desempenho e escolhendo próximos pontos a testar com base em aquisições).

Há também algoritmos evolucionários e de otimização de múltiplos começos. Em resumo, escolher modelos/hiperparâmetros é um problema de otimização de caixa-preta, onde cada avaliação (treinar/validar um modelo) pode ser custosa e precisamos ser espertos em quantas avaliações fazemos.

Validação cruzada e suas variações

O método K-fold que aplicamos é um dos preferidos dos(as) cientistas de dados pela eficiência e simplicidade. Tipicamente usa-se $K=5$ ou $K=10$. Mas há cenários e variações a considerar:

- **Hold-out simples:** é o clássico train/test split. Você reserva, digamos, 20% dos dados para teste (ou validação) e usa 80% para treino. É menos custoso computacionalmente que K-fold (pois treina apenas 1 vez em vez de K vezes), porém a estimativa de desempenho pode ter maior variância dependendo de como os dados foram divididos. Uma única divisão pode, por acaso, favorecer um modelo em detrimento de outro (talvez a divisão A continha mais outliers no teste). O K-fold ameniza isso ao “rodar várias divisões”. Ainda assim, o hold-out é válido, especialmente quando o volume de dados é grande o suficiente para que uma parte isolada já represente bem o todo.
- **Leave-One-Out (LOO):** é um caso extremo de K-fold onde $K = N$ (número de exemplos). Ou seja, treinamos o modelo N vezes, cada vez deixando apenas um exemplo para validação. O LOO aproveita maximamente os dados de treino em cada rodada (usa $N-1$ exemplos de N) e produz uma estimativa de generalização geralmente sem viés, mas pode ter variância alta e ser computacionalmente caro (treinar N vezes). Ele é útil em contextos com pouquíssimos dados, mas muitas vezes $K=5$ ou $K=10$ atinge um bom compromisso.
- **Validação Estratificada:** quando há distribuição de classes desbalanceada, convém dividir os folds preservando a proporção de classes. Por exemplo, em um dataset de classificação binária com 90% dos exemplos da classe 0 e 10% da classe 1, um K-fold aleatório pode, por sorte, gerar um fold de validação quase sem exemplos da classe

minoritária, distorcendo a avaliação. A validação cruzada estratificada garante que cada fold tenha aproximadamente 90/10 de classes, espelhando o todo. Isso reduz a variância nas métricas e evita casos extremos de folds não representativos.

- **Validação em Séries Temporais:** quando os dados têm dependência temporal (por exemplo, previsões de vendas ao longo do tempo), não podemos simplesmente embaralhar e dividir, pois isso quebraria a correlação temporal. A validação cruzada para séries temporais segue outro esquema: normalmente separa blocos ordenados no tempo para treinamento e validação, respeitando a cronologia (às vezes chamada blocked cross-validation). Um método comum é a validação em blocos temporais deslizantes: por exemplo, usar dados até mês X para treinar e o mês X+1 para validar, repetindo isso ao longo da série. Outra técnica é a gap validation, que deixa um “gap” de tempo entre treino e validação para evitar autocorrelação imediata. O importante é não misturar passado e futuro nos folds – o fold de validação deve ser sempre posterior aos dados de treino naquele cenário.
- **Repetição e média de resultados:** podemos repetir o K-fold várias vezes com partições diferentes (especialmente se N for pequeno). Por exemplo, 10x10-fold CV: roda 10-fold CV dez vezes com divisões aleatórias distintas e faz a média. Isso diminui a variância da estimativa, embora seja caro. Em competições de data science, por exemplo, às vezes se faz repeated stratified K-fold para obter uma avaliação bem estável.
- **Nested cross-validation:** é uma técnica importante quando estamos ao mesmo tempo selecionando hiperparâmetros e medindo a performance final. Por exemplo, imagine que usamos 5-fold CV para achar o melhor grau de polinômio. Teremos escolhido um modelo no fim; se agora reportamos o erro médio de 5-fold obtido, estaremos ligeiramente otimistas, pois aquele erro médio foi usado na seleção. A solução é aninhar: uma camada externa de CV para avaliar generalização e, dentro de cada treino da camada externa, rodar outra CV (interna) para selecionar hiperparâmetros. Isso é computacionalmente pesado, mas dá

uma avaliação imparcial. Em problemas reais, às vezes se simplifica: separa-se os dados em três partes: treino, validação e teste finais. Usa-se treino+validação (via CV interna) para achar o melhor modelo e depois confirma-se o desempenho em um conjunto de teste final que ficou guardado sem ser tocado até o fim. Essa abordagem de train/val/test é análoga ao Blue/Green deployment com um “ambiente de stage”: garante que temos pelo menos uma avaliação final totalmente isenta de viés de seleção.

Testes estatísticos e significância de diferenças

Ao comparar modelos, principalmente quando seus desempenhos são próximos, é prudente perguntar: a diferença observada é estatisticamente significativa? Por exemplo, suponha que o Modelo A deu 0.85 de acurácia média em CV e o Modelo B deu 0.83. Parece que A é melhor – mas e se isso foi sorte das amostras? Ferramentas estatísticas como o teste t pareado ou o teste de Wilcoxon podem ser aplicadas sobre os resultados dos K folds de CV para verificar se, por exemplo, μ_A teve erro menor que μ_B de forma consistente (hipótese nula: “os modelos têm desempenho igual”).

Dietterich (1998) sugere um teste 5x2 cv (5 repetições de 2-fold) combinado com um teste t especial para comparar dois algoritmos de aprendizado, como forma robusta de medir significância. Já para classificadores comparando acertos/erros, podemos usar o teste de McNemar em resultados pareados de validação (especialmente quando usamos apenas um conjunto de validação fixo). Em termos menos formais, muitos praticantes adotam uma regra simples: se a diferença de desempenho for menor que a variância (ou um múltiplo do desvio-padrão) das medições de validação, considere que não há diferença clara. Por exemplo, se na CV de 10 folds o modelo X teve acurácia 0.90 ± 0.02 e o Y teve 0.88 ± 0.03 , as distribuições se sobrepõem; talvez valha escolher o mais simples.

Essa é a ideia por trás da regra do desvio-padrão em seleção de modelos: escolher o modelo mais simples cujo desempenho esteja a no máximo 1 desvio-padrão do desempenho do melhor modelo. Essa heurística (às vezes chamada 1-SE

rule) é mencionada por Hastie et al. (2009) ao selecionar a árvore ótima na poda de uma árvore de decisão, por exemplo. Em suma, a validação cruzada nos dá números, mas precisamos interpretá-los com cautela – diferenças pequenas podem não ser confiáveis.

Hiperparâmetros e validação: early stopping, regularização e outros truques

Até agora falamos de escolher entre modelos distintos (por exemplo, polinômios de graus diferentes, ou algoritmos diferentes). Mas a linha entre “modelo” e “hiperparâmetro” às vezes é tênue. Hiperparâmetros são ajustes do próprio algoritmo que não são aprendidos durante o treino supervisionado usual. A escolha deles pode ser vista como seleção de modelos dentro de uma família contínua. Por exemplo, a taxa de regularização (λ) em uma regressão Ridge controla o trade-off viés/variância; cada valor de λ define de fato um modelo diferente (com parâmetros ótimos diferentes). Fazemos validação para escolhê-la.

Outro exemplo é o número de épocas de treino de uma rede neural: treinar mais épocas pode levar a sobreajuste; poucas, a underfitting. O early stopping lida com isso usando parte do treinamento como validação: o algoritmo observa o erro de validação a cada época e para quando esse erro começa a piorar (ou deixa de melhorar) significativamente. Trata-se, essencialmente, de usar um pequeno hold-out dentro do treinamento para escolher o hiperparâmetro “número de épocas” de forma dinâmica.

Nesse caso, não estamos usando K-fold, mas o princípio é o mesmo: manter um conjunto de validação (tipicamente 5-20% dos dados de treino) para monitorar desempenho e então selecionar o modelo obtido na época onde o erro de validação mínimo ocorreu. Early stopping é amplamente utilizado no treinamento de deep learning para evitar sobreajuste sem ter que manualmente decidir o número de iterações. Outra faceta: regularização (L2, L1, dropout etc.) e data augmentation são técnicas que visam reduzir sobreajuste e a intensidade com que as aplicamos costuma ser hiperparâmetro também (ex.: peso de decaimento L2, probabilidade de dropout).

Mais uma vez, definimos esses valores via validação. Podemos até encarar a arquitetura da rede (quantas camadas/neurônios) como hiperparâmetros a serem

validados. Hoje existem abordagens de AutoML que fazem otimização de arquitetura (Neural Architecture Search) e hiperparâmetros de forma automática. Contudo, mesmo métodos automáticos usam validação interna para avaliar cada candidato. No fim, o conceito universal é: use sempre dados não vistos pelo modelo para guiá-lo nas escolhas, seja escolha de pesos (treinamento) ou de hiperparâmetros/arquitetura (validação). Ignorar isso leva a modelos que “colam” no conjunto de treino e fraquejam fora dele.

Custos computacionais e viabilidade

É importante comentar que validação cruzada adiciona custo computacional. Se treinamos um modelo em 1 hora, uma 5-fold CV desse modelo levará ~5 horas (5 treinamentos). Se estamos fazendo busca em hiperparâmetros, multiplique pelo número de configurações testadas. Isso pode ficar pesado. Em projetos reais, deve-se equilibrar rigor e custo: para conjuntos de dados muito grandes, às vezes opta-se por um split único (talvez repetido algumas vezes) em vez de CV completa, para economizar tempo. Em competições ou pesquisa, onde cada fração de performance importa, gasta-se mais: por exemplo, não é incomum times de Kaggle treinarem dezenas de modelos e usarem 10-fold CV repetido para cada, gastando uma quantidade considerável de computação em nuvem.

Hoje existem plataformas de AutoML e frameworks de tuning que distribuem essas tarefas em múltiplas máquinas. Também é possível usar técnicas de amostragem do conjunto de treinamento para estimar rapidamente se um hiperparâmetro parece promissor, antes de treinar no full dataset – tomando decisões mais rápidas e refinando incrementalmente (por exemplo, treinar em 20% dos dados para filtrar hiperparâmetros ruins e treinar em 100% só os melhores). Além disso, podemos reaproveitar modelos entre folds parcialmente: alguns algoritmos permitem warm-start, começando o treinamento do próximo fold a partir de um modelo treinado em fold anterior, com ajustes, embora nem sempre isso se aplique facilmente.

Apesar do custo, a validação cruzada é considerada um padrão-ouro no benchmarking de modelos. Em muitas aplicações, treinar 5x ou 10x não é inviável (quando o dataset ou modelo não é gigante). E com recursos como

multiprocessamento, podemos rodar folds em paralelo. Ou seja, o custo é linear em K , mas K é moderado e pode haver paralelismo. O resultado – uma estimativa confiável de qual modelo/ajuste é melhor – geralmente compensa o tempo gasto, evitando apostar no modelo errado em produção (o que poderia ter um custo muito maior!).

Governança e boas práticas na seleção de modelos

Em times profissionais de Machine Learning, a seleção de modelos não fica solta sem controle. Assim como em DevOps há políticas para aprovar deploys, em MLOps existem políticas para garantir que o modelo escolhido passou por validações adequadas. Por exemplo, pode haver uma exigência de que todo modelo novo seja avaliado em um conjunto de teste separado (inacessível aos desenvolvedores, justamente para evitar qualquer viés involuntário) e alcance pelo menos a métrica dos modelos atuais em produção. Algumas empresas mantêm um comitê de modelos: um grupo que revisa os experimentos, verifica se não houve vazamento de dados (data leakage) – situação em que acidentalmente informações do teste vazam para o treino, dando uma impressão inflada de performance.

Vazamentos podem ocorrer de formas sutis, como normalizar atributos usando média/desvio de todo o dataset antes de separá-lo (a informação do teste contamina o treino). Por isso, uma regra é: todo pré-processamento de dados deve ser feito dentro do ciclo de validação, nunca antes (ex.: ao usar K-fold, normalizar cada fold de treino com média e desvio daquele fold, não do dataset completo). Esses cuidados são muitas vezes codificados em checklists ou até automatizados em pipelines de experimentos.

Ferramentas de gerenciamento de experimentos, como MLflow e plataformas cloud (Azure ML, AWS SageMaker, Google Vertex AI), permitem registrar cada experimento com suas configurações, métricas de validação e arquivos de modelo. Isso facilita auditorias e reprodutibilidade. Por exemplo, podemos consultar “qual foi o AUC obtido por tal modelo em sua validação cruzada?” ou até reproduzir o treinamento com a mesma seed aleatória. Essa reprodutibilidade é essencial: se um modelo foi escolhido para produção, precisamos confiar que ele é resultado de um

processo robusto, não de um acaso irreprodutível. Em órgãos regulados (finanças, saúde), guardar evidências de validação é parte da compliance.

Outra boa prática de governança é definir baseline e objetivos: aqui definimos baseline models (ex.: “modelo atual em produção tem 85% de acurácia”) e targets (“queremos pelo menos 88% com novo modelo”). A validação robusta nos dirá se atingimos isso. Se um novo modelo supercomplexo traz 0.5% de ganho, vale a pena? Às vezes modelos mais simples são preferíveis se o ganho não for significativo, pois são mais interpretáveis ou rápidos. O caso do Netflix Prize ilustra bem: a combinação de centenas de modelos deu só um pouquinho a mais de acurácia e foi preterida por uma solução mais simples.

Em resumo, a seleção de modelos não é “terra de ninguém”: envolve metodologia (validação apropriada), gerenciamento de experimentos e critério para julgar resultados. Assim, garantimos que a escolha final – seja um modelo para colocar em produção ou publicar em um paper – foi baseada em evidências sólidas e comparações justas.

MERCADO, CASES E TENDÊNCIAS

A aplicação cuidadosa de seleção de modelos e validação cruzada não fica só nos livros – ela influencia decisões reais em empresas e projetos famosos. Vamos ver alguns casos (reais ou inspirados em eventos reais) que ressaltam lições sobre a importância de uma boa validação, bem como tendências de mercado relacionadas a automação e melhores práticas no desenvolvimento de modelos de ML.

Na figura a seguir, os dados são divididos em $K=5$ folds; em cada iteração, um fold (cor rosa) serve de validação enquanto os outros (azul) são usados para treino. Após 5 iterações, obtém-se um conjunto de 5 medidas de desempenho que são então agregadas (média, desvio-padrão etc.) para estimar a performance do modelo.



Figura 1 – Esquema ilustrativo da validação cruzada K-fold
Fonte: Google Imagens (2026)

Netflix Prize (Recomendação) – Um caso clássico que evidencia a necessidade de equilibrar desempenho e complexidade. Entre 2006 e 2009, a Netflix promoveu um concurso público desafiando times a melhorarem em 10% a acurácia de seu algoritmo de recomendação de filmes. O time vencedor alcançou um impressionante ganho de 10.06% no RMSE usando um ensemble de centenas de modelos. Entretanto, a Netflix nunca implantou diretamente esse modelo campeão. Motivo? Ele era extremamente complexo e os ganhos adicionais não justificavam o esforço de engenharia para produção. Em outras palavras, embora a métrica de

validação (no caso, um teste em dados históricos dividido pela própria Netflix) tenha sido otimizada ao máximo – provando que o modelo podia ser ligeiramente melhor – a empresa avaliou que a melhoria (~0.03 absolutos em RMSE) não era significativa o suficiente frente ao custo e à mudança de foco para novos desafios de recomendação.

Esse caso ilustra dois pontos. Primeiro: a validação (neste caso, o hold-out grande da Netflix) foi bem-feita; ela realmente provou que o ensemble gigante era melhor em dados novos. Ou seja, não foi um caso de overfitting ao teste – a própria Netflix confirmou offline o ganho. Porém, a métrica não é tudo: fatores como simplicidade, tempo de resposta e manutenção pesam. Em muitos cenários corporativos, um modelo 5% pior, porém mais simples, pode ser preferível. A Netflix optou por modelos mais simples e rápidos de atualizar, priorizando experiência do usuário e outros produtos (como streaming). Essa história, famosa no meio, reforça uma boa prática: sempre considerar se um modelo mais complexo traz ganho suficiente para valer a pena. Ferramentas de validação nos dão a resposta técnica (modelo A é estatisticamente melhor que B?), mas a decisão de adoção envolve também julgamento de valor prático.

Kaggle e Competição de Modelos – No mundo das competições de Data Science, validação cruzada é lei. Participantes de alto nível tipicamente treinam e validam seus modelos com extremo rigor para evitar serem enganados pelo public leaderboard (resultado em um subconjunto de teste público). É comum eles usarem, por exemplo, 5-fold CV local e só submeterem ao leaderboard a média ou ensemble das predições dos 5 modelos. Assim, reduzem o risco de overfitting no leaderboard público. Ainda assim, casos de “shake-up” acontecem: times que estavam no topo do ranking público caem muito no ranking final privado, evidenciando que acabaram adaptando demais suas soluções ao conjunto de teste visível. As equipes vencedoras quase sempre salientam a importância de uma validação robusta.

Por exemplo, na competição do Zillow Prize (estimativa de valores residenciais), o vencedor relatou ter usado validação cruzada estratificada por região geográfica para garantir que seu modelo generalizava bem, evitando sobreajuste a regiões específicas. Outro caso: em competições de visão computacional, participantes criam seus próprios conjuntos de validação (às vezes coletam dados extras rotulados) para ter mais confiança. A cultura do Kaggle disseminou várias

técnicas de validação, incluindo formas de blending/stacking que envolvem validação (como treinar modelos de nível 2 apenas com previsões out-of-fold de modelos nível 1, uma técnica sofisticada que impõe que nenhum modelo veja as previsões de outro nos mesmos dados de treino sem validação cruzada). Em resumo, nas competições o mantra é: “valide como se sua posição dependesse disso” – e ela de fato depende.

Startup X (Fiasco por Falta de Validação) – Nem todo mundo segue as boas práticas... considere uma startup fictícia (mas inspirada em histórias reais) que desenvolve um modelo para identificar clientes propensos a cancelar um serviço (churn). Animados, os(as) cientistas de dados escolhem um algoritmo de rede neural extremamente complexo e o treinam no histórico de clientes. Internamente, eles observam métricas de treino altíssimas – acurácia de 98%! –, muito melhor que modelos mais simples que davam ~85%. Sem fazer validação adequada, apresentam o modelo “quase perfeito” para a diretoria, que decide implementá-lo em produção. Três meses depois, percebe-se que o modelo mal acerta quem vai cancelar; a taxa de retenção não melhorou. O que houve? Ao investigar, notam que o modelo memorizou padrões específicos do conjunto de treino (por exemplo, identificou códigos de cliente ou datas particulares correlacionadas ao churn na amostra histórica, mas que não se repetem no futuro).

Em suma, um clássico overfitting. Se tivessem feito uma validação cruzada ou pelo menos um hold-out, teriam visto que a acurácia despencava para ~80-85%, sinal de que o ganho aparente era ilusório. Esse fiasco hipotético ecoa casos verídicos onde empresas pulam etapas de validação por excesso de confiança ou pressa. As lições: nunca confie apenas no desempenho em treino; sempre valide em dados separados. Modelos muito poderosos sem validação são como “mísseis guiados pela mão do atirador” – eles acertam o alvo de treino porque o viram, mas erram o alvo real. Infelizmente, histórias assim existem e levaram organizações a institucionalizar checagens: por exemplo, exigir que qualquer métrica reportada venha de um conjunto de validação. Hoje, até mesmo ferramentas de AutoML simples, como do scikit-learn, fazem automaticamente uma divisão treino/validação para evitar que usuários inexperientes se enganem com scores de treino.

MLOps e Automatização (Google, etc.) – Em empresas de tecnologia mais avançadas, todo o processo de seleção de modelos vem se tornando automatizado

e integrado ao ciclo de desenvolvimento. A Google, por exemplo, desenvolveu o Google Vizier, um serviço interno (hoje também disponível na Google Cloud como Vertex AI Vizier) para otimização de hiperparâmetros e validação automatizada. Os engenheiros podem lançar experimentos que o sistema distribui em diversos workers, testando configurações de modelos e calculando métricas de validação (usando métodos bayesianos para guiar os testes seguintes). O resultado é que, em vez de manualmente tentar hiperparâmetros, o Vizier retorna a melhor configuração encontrada e seu desempenho validado. Isso acelera muito a experimentação – um trabalho que podia levar semanas manualmente pode ser feito em dias ou horas, permitindo que cientistas se concentrem em aspectos de arquitetura e interpretação.

Microsoft e AWS oferecem serviços similares de Hyperparameter Tuning que rodam n treinos em paralelo e até param aqueles com performance claramente inferior (para economizar recursos). Outra tendência strong é a integração contínua de modelos: assim como se faz CI/CD de software, faz-se CI/CD de modelos. Por exemplo, o Facebook criou pipelines onde, ao surgir um novo conjunto de dados ou features, um treinamento + validação cruzada é disparado automaticamente; se o novo modelo supera o atual por uma margem X, ele é sinalizado para possível implantação. Isso só é possível graças a automação das validações – as métricas de validação funcionam como “testes unitários” do modelo. De fato, algumas empresas tratam certos critérios de validação como gating: “só promover para produção se a AUC_{vl} for pelo menos 1 ponto maior que do modelo vigente”. Essa abordagem lembra muito o Canary release ou blue/green do DevOps, mas focado em modelos: libera o novo em produção apenas se a validação offline indica que é seguro.

AutoML e Democratização – Nos últimos anos, surgiu uma forte tendência de ferramentas de AutoML que prometem construir e validar modelos automaticamente, sem intervenção humana intensa. AutoML frameworks (AutoKeras, H2O Driverless AI, TPOT, Auto-sklearn etc.) utilizam validação cruzada interna para comparar múltiplos algoritmos (árvores, redes, SVMs, ...) e buscar hiperparâmetros, retornando o modelo “ótimo” segundo uma métrica. Empresas como DataRobot popularizaram esse conceito para que times sem expertise profunda em ML pudessem ainda assim ter modelos decentes, com validação cuidada pela plataforma. É claro que AutoML não elimina a necessidade de senso

crítico: ele pode devolver um modelo com ótima métrica, mas cabe aos especialistas avaliar se faz sentido e se atinge os objetivos de negócio.

Mesmo assim, essas ferramentas incorporam em seus bastidores muito do que discutimos: fazem várias splits de dados, usam até nested CV para evitar viés e produzem relatórios de leaderboard com diferentes modelos e seus scores de validação. A tendência é que aspectos técnicos de tuning fiquem cada vez mais automáticos, de modo que humanos possam focar em definições de problema, coleta de dados e interpretação dos resultados – coisas que AutoML não resolve. Em cenário de mercado, isso significa democratização do desenvolvimento de modelos: empresas menores ou equipes com menos cientistas PhD conseguem ainda assim treinar modelos competitivos usando ferramentas autoassistidas (desde que tenham dados de qualidade).

Monitoramento contínuo e revalidação – Por fim, vale notar uma tendência emergente pós-implantação: uma vez que o modelo está em produção, o trabalho não termina. Cada previsão feita gera novos dados (ex.: você previu que um cliente vai cancelar e ele de fato cancelou ou não; isso é feedback). Práticas modernas de MLOps prevêm captar esse feedback e periodicamente revalidar o modelo. Isso geralmente assume a forma de retreinar o modelo com dados novos e/ou realizar testes de desempenho periódicos. Um conceito relevante é o de drift: se a distribuição dos dados de entrada muda ao longo do tempo (por exemplo, comportamento de usuários mudando), o modelo pode perder acurácia. Algumas ferramentas monitoram estatísticas dos dados em produção e disparam alertas se desviam muito do treinamento original – indicando potencial necessidade de revalidar.

Em certas operações críticas, empresas fazem um shadow deployment: o modelo novo (retreinado) roda em paralelo, recebendo tráfego real, mas suas decisões não impactam o usuário – servem apenas para coletar métricas que depois são comparadas ao modelo atual. Isso fornece uma validação em ambiente real antes de trocar de modelo (similar a um canary, só que as decisões não são aplicadas). Se o desempenho for consistentemente melhor, aí sim o modelo novo assume. Esse ciclo contínuo assegura que, mesmo após a entrega, estamos validando e selecionando modelos de forma adaptativa.

Em suma, a seleção de modelos e a validação cruzada evoluíram de técnicas manuais em projetos isolados para componentes integrados nos fluxos de trabalho de Machine Learning das empresas. A cultura de dados hoje reconhece que não basta construir um modelo brilhante – é preciso provar sua robustez antes, durante e depois da implantação. Ferramentas e tendências de mercado refletem isso: desde casos emblemáticos que viraram lição, até plataformas que automatizam validação, tudo converge para reduzir o risco de modelos mal validados causando prejuízos (seja financeiro, seja de reputação). Afinal, um modelo de IA em produção é tão bom quanto seus resultados em dados reais – e garantir que ele alcance esses resultados exige todo um cuidado no processo de seleção e validação, exatamente o tema desta aula.

O QUE VOCÊ VIU NESTA AULA?

Ufa! Percorremos bastante terreno. Vamos recapitular de forma leve os principais aprendizados.

Entendemos o problema do overfitting vs underfitting: modelos muito complexos podem enganar, indo muito bem nos dados de treino e fracassando em generalizar. Vimos que separar dados para validação é essencial para estimar o desempenho real do modelo – assim como em desenvolvimento de software usamos ambiente de teste para evitar surpresas em produção. No hands on, aplicamos 5-fold CV para escolher o grau de um polinômio que melhor se ajusta a um conjunto sintético, evitando sobreajuste. Também vimos variações: LOOCV (deixar-um-de-fora) como caso extremo, validação estratificada para manter proporções, cuidados especiais para séries temporais (não misturar passado/futuro nos folds) e até métodos de validação repetida para estimativas mais estáveis. Também ressaltamos a importância de nunca usar dados de teste na etapa de treino/validação (evitando vazamento de dados).

Exploramos como hiperparâmetros (ex.: regularização, profundidade de árvore, taxa de aprendizagem) são escolhidos via validação. Falamos de abordagens de busca – de grid search exaustivo a random search e métodos bayesianos – e observamos que random search pode ser mais eficiente em muitos cenários. Vimos que algumas heurísticas, como early stopping, internalizam a validação para interromper treino quando o modelo começa a sobreajustar em um conjunto de validação. Abordamos também o custo computacional: validar K-fold vezes e experimentar muitos hiperparâmetros requer poder de processamento; entretanto, argumentamos que esse custo é justificado para evitar erros graves de seleção.

Destacamos que, assim como existem pipelines de CI/CD em software, existem pipelines de experimentação em ML. Falamos sobre manter reprodutibilidade (logar seeds, guardar conjuntos de validação usados) e documentar resultados. Conceitos como data leakage foram enfatizados e como detectá-los/evitá-los. Introduzimos a ideia da regra do 1 desvio-padrão (ou 1-SE)

para optar por modelos mais simples caso a diferença seja marginal, trazendo uma perspectiva prática de não buscar complexidade à toa.

Em conclusão, você deve ter consolidado a noção de que selecionar um modelo não é um chute nem uma decisão arbitrária – é um processo científico e iterativo, amparado por técnicas de validação. Aprender a usar adequadamente a validação cruzada e outras ferramentas garante que suas escolhas de modelo sejam baseadas em evidências sólidas, reduzindo muito as chances de surpresas desagradáveis quando seu modelo encarar dados reais.

Essa postura rigorosa separa soluções de ML confiáveis de “curvas ajustadas demais” fadadas ao fracasso. Com o conhecimento desta aula, esperamos que você se sinta pronto(a) para projetar experimentos de aprendizado de máquina de forma mais segura: definindo bons esquemas de validação, interpretando resultados com senso crítico e, assim, escolhendo modelos com confiança e embasamento.

REFERÊNCIAS

- ARLOT, S.; CELISSE, A. A survey of cross-validation procedures for model selection. **Statistics Surveys**, 4: 40–79. 2010. Disponível em: <https://projecteuclid.org/journals/statistics-surveys/volume-4/issue-none/A-survey-of-cross-validation-procedures-for-model-selection/10.1214/09-SS054.full>. Acesso em: 23 fev. 2026.
- BERGSTRA, J.; BENGIO, Y. Random Search for Hyper-Parameter Optimization. **Journal of Machine Learning Research**, 13(10): 281–305. 2012. Disponível em: <https://jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>. Acesso em: 23 fev. 2026.
- BISHOP, C. M. **Pattern Recognition and Machine Learning**. 2006. Disponível em: <https://link.springer.com/book/9780387310732>. Acesso em: 23 fev. 2026.
- BROWNLEE, J. **A Gentle Introduction to k-fold Cross-Validation**. 2023. Disponível em: <https://machinelearningmastery.com/k-fold-cross-validation>. Acesso em: 23 fev. 2026.
- DIETTERICH, T. G. **Approximate statistical tests for comparing supervised classification learning algorithms**. 1998. Disponível em: <https://direct.mit.edu/neco/article-abstract/10/7/1895/6224/Approximate-Statistical-Tests-for-Comparing?redirectedFrom=fulltext>. Acesso em: 23 fev. 2026. Acesso em: 23 fev. 2026.
- GÉRON, A. **Hands-On Machine Learning with Scikit-Learn & TensorFlow**. 2. ed.). [s.l.]: O'Reilly, 2019.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. [s.l.]: MIT Press, 2016.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning**. 2. ed. [s.l.]: Springer, 2009.
- KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: **Proc. of IJCAI**. 1995. Disponível em: <https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf>. Acesso em: 23 fev. 2026.
- RASCHKA, S. **Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning**. 2018. Disponível em: <https://arxiv.org/abs/1811.12808>. Acesso em: 23 fev. 2026.
- YAU, N. **Why \$1M Netflix algorithm never went to production**. 2012. Disponível em: <https://flowingdata.com/2012/04/17/why-1m-netflix-algorithm-never-went-to-production>. Acesso em: 23 fev. 2026.

PALAVRAS-CHAVE

Overfitting. Validação cruzada. Hiperparâmetros.

EMAP



POSTECH